



DiOMP-Offloading: Toward Portable Distributed Heterogeneous OpenMP

Baodi Shan¹, Mauricio Araya-Polo², Barbara Chapman¹

1 Stony Brook University; 2 TotalEnergies

Nov 16, 2025

Background: OpenMP & OpenMP Offloading

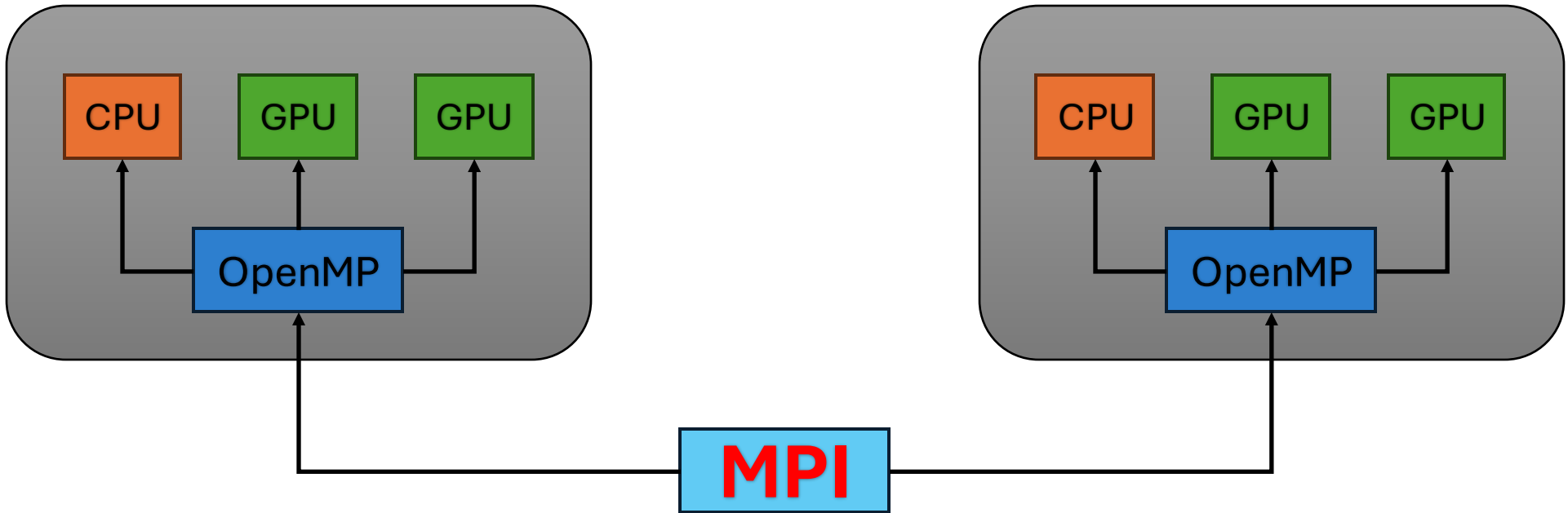
```
//foo.c
int a[10], b[10], c[10];
#pragma omp parallel for
for(int i = 0; i < 10; ++i)
    c[i] = a[i] + b[i]
```

```
clang foo.c -fopenmp -o foo
```

```
//foo.c
int a[10], b[10], c[10];
#pragma omp target teams distribute parallel for
for(int i = 0; i < 10; ++i)
    c[i] = a[i] + b[i]
```

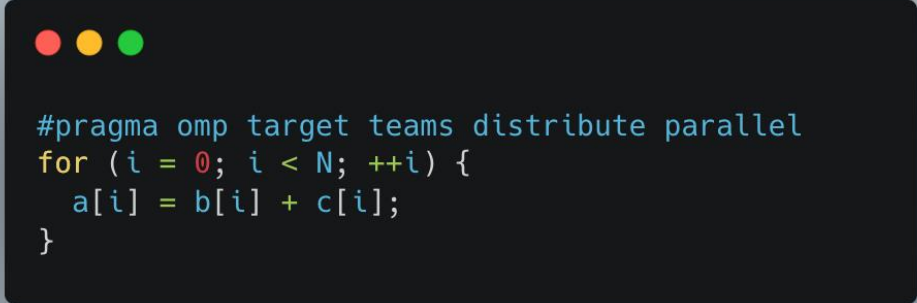
```
clang foo.c -fopenmp -offload-arch=sm_80 -o foo
```

Background: MPI + OpenMP Offloading



OpenMP Offloading + MPI RMA?

- **Separated Workflows:**
 - **OpenMP (libomptarget):** Manages device memory via its own data map table, handling both *implicit* (in-kernel usage) and explicit (target data map) data motion.
 - **MPI (RMA):** Requires a separate, explicit memory management mechanism (e.g., MPI_Win) layered on top of the same device memory.
- **Consequences:**
 - Both layers access GPU memory independently, leading to data and metadata redundancy.
 - **Inconsistent Synchronization:** Must manage both OpenMP implicit barriers and MPI fences, which is error-prone and inefficient.



```
#pragma omp target teams distribute parallel
for (i = 0; i < N; ++i) {
    a[i] = b[i] + c[i];
}
```

OpenMP Offloading + MPI RMA?

- **Separated Workflows:**

- **OpenMP (libomptarget):** Manages device memory via its own data map table, handling both implicit (in-kernel usage) and **explicit** (target data map) data motion.
- **MPI (RMA):** Requires a separate, explicit memory management mechanism (e.g., MPI_Win) layered on top of the same device memory.

- **Consequences:**

- Both layers access GPU memory independently, leading to data and metadata redundancy.
- **Inconsistent Synchronization:** Must manage both OpenMP implicit barriers and MPI fences, which is error-prone and inefficient.

```
#pragma omp target teams distribute parallel for \
    map(to: b[0:N], c[0:N]) \
    map(from: a[0:N])
for (i = 0; i < N; ++i) {
    a[i] = b[i] + c[i];
}
```

```
void *dptr = omp_get_mapped_ptr(a, dev);

MPI_Win win;
MPI_Win_create(dptr, N*sizeof(TYPE), sizeof(TYPE),
    \ MPI_INFO_NULL, MPI_COMM_WORLD, &win);
```

OpenMP Offloading + MPI RMA?

- **Separated Workflows:**
 - **OpenMP (libomptarget):** Manages device memory via its own data map table, handling both implicit (in-kernel usage) and **explicit** (target data map) data motion.
 - **MPI (RMA):** Requires a separate, explicit memory management mechanism (e.g., MPI_Win) layered on top of the same device memory.
- **Consequences:**
 - Both layers access GPU memory independently, leading to data and metadata redundancy.
 - **Inconsistent Synchronization:** Must manage both OpenMP implicit barriers and MPI fences, which is error-prone and inefficient.

```
omp_target_alloc(a, N, device_id);
```

```
MPI_Win_create(a, N*sizeof(TYPE), sizeof(TYPE), \
               MPI_INFO_NULL, MPI_COMM_WORLD, &win);
```

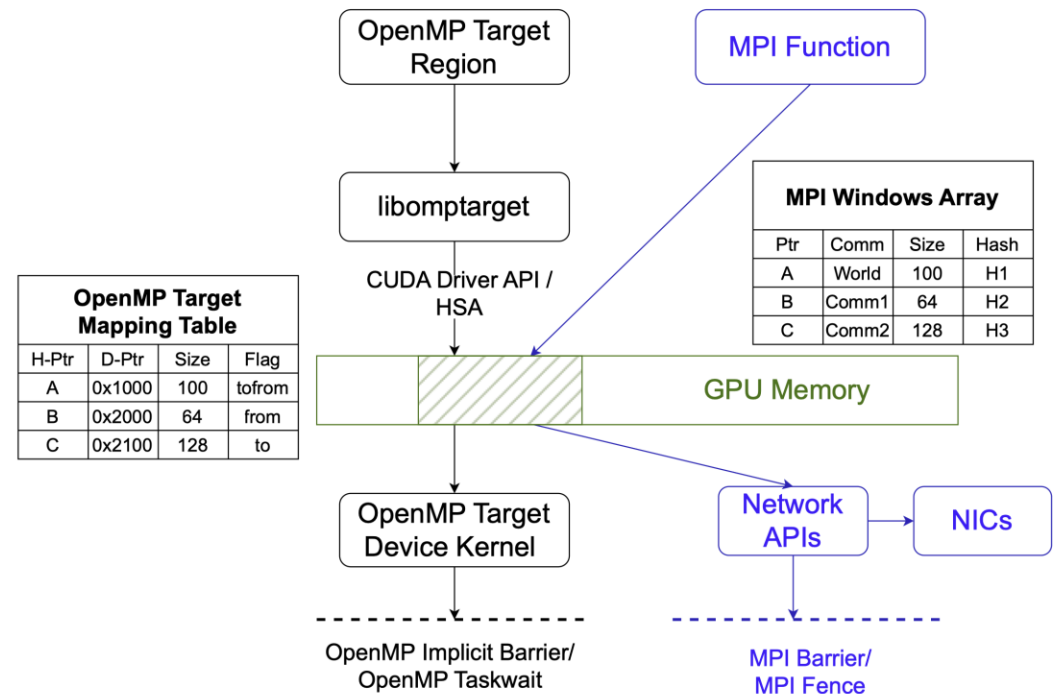
OpenMP Offloading + MPI RMA?

- **Separated Workflows:**

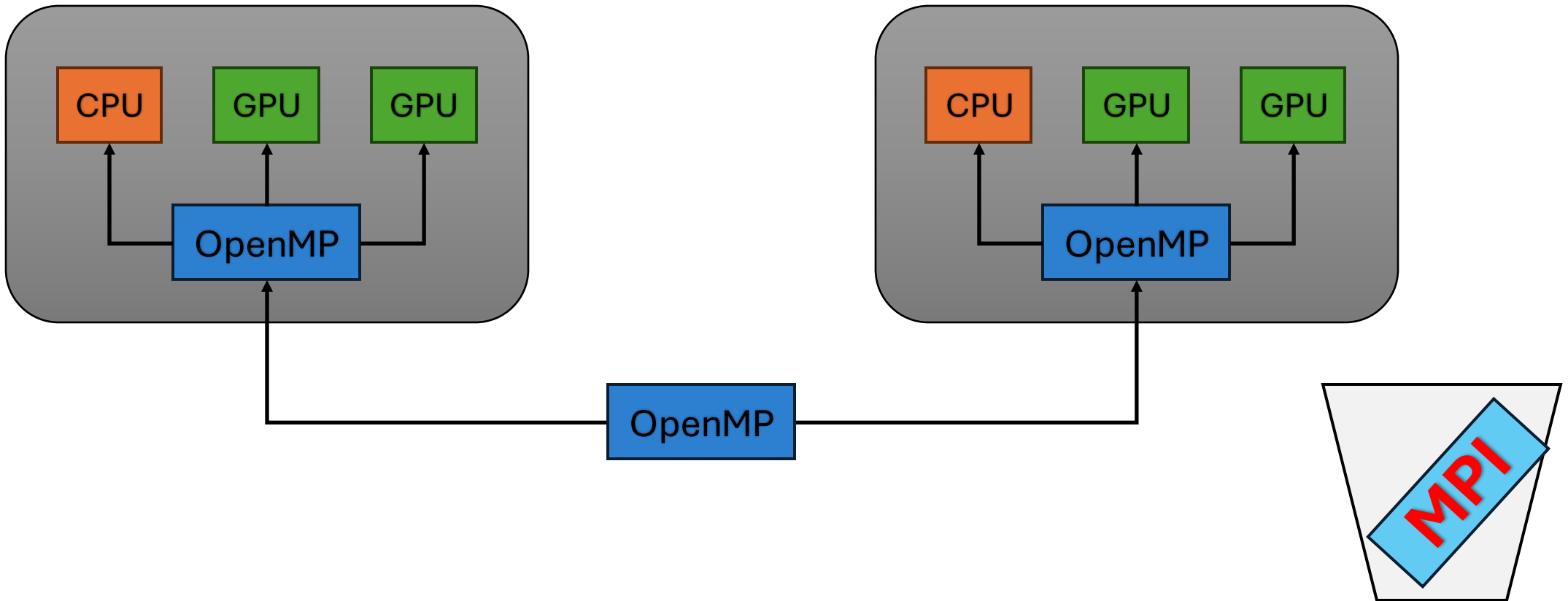
- **OpenMP (libomptarget):** Manages device memory via its own data map table, handling both implicit (in-kernel usage) and explicit (target data map) data motion.
- **MPI (RMA):** Requires a separate, explicit memory management mechanism (e.g., MPI_Win) layered on top of the same device memory.

- **Consequences:**

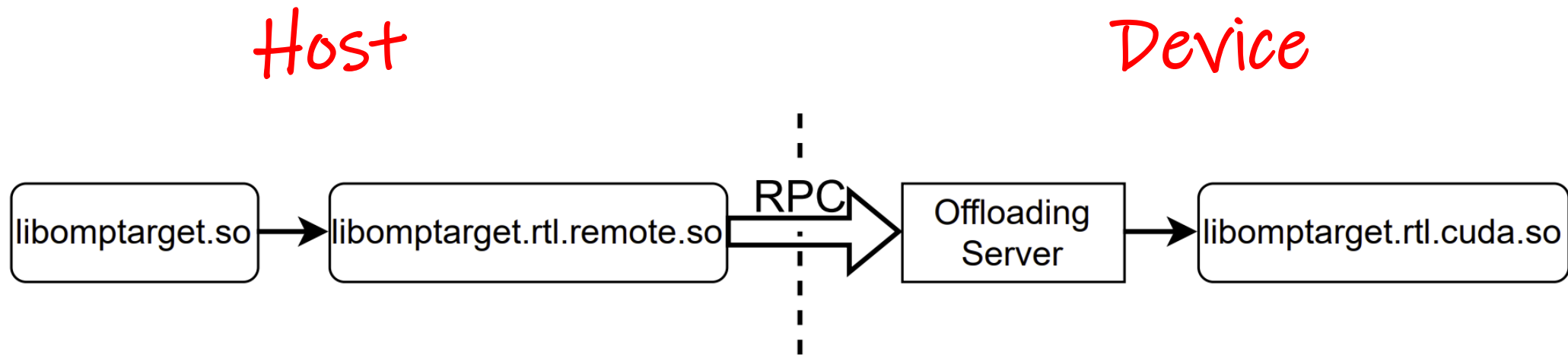
- Both layers access GPU memory independently, leading to data and metadata redundancy.
- **Inconsistent Synchronization:** Must manage both OpenMP implicit barriers and MPI fences, which is error-prone and inefficient.



Background: MPI+OpenMP

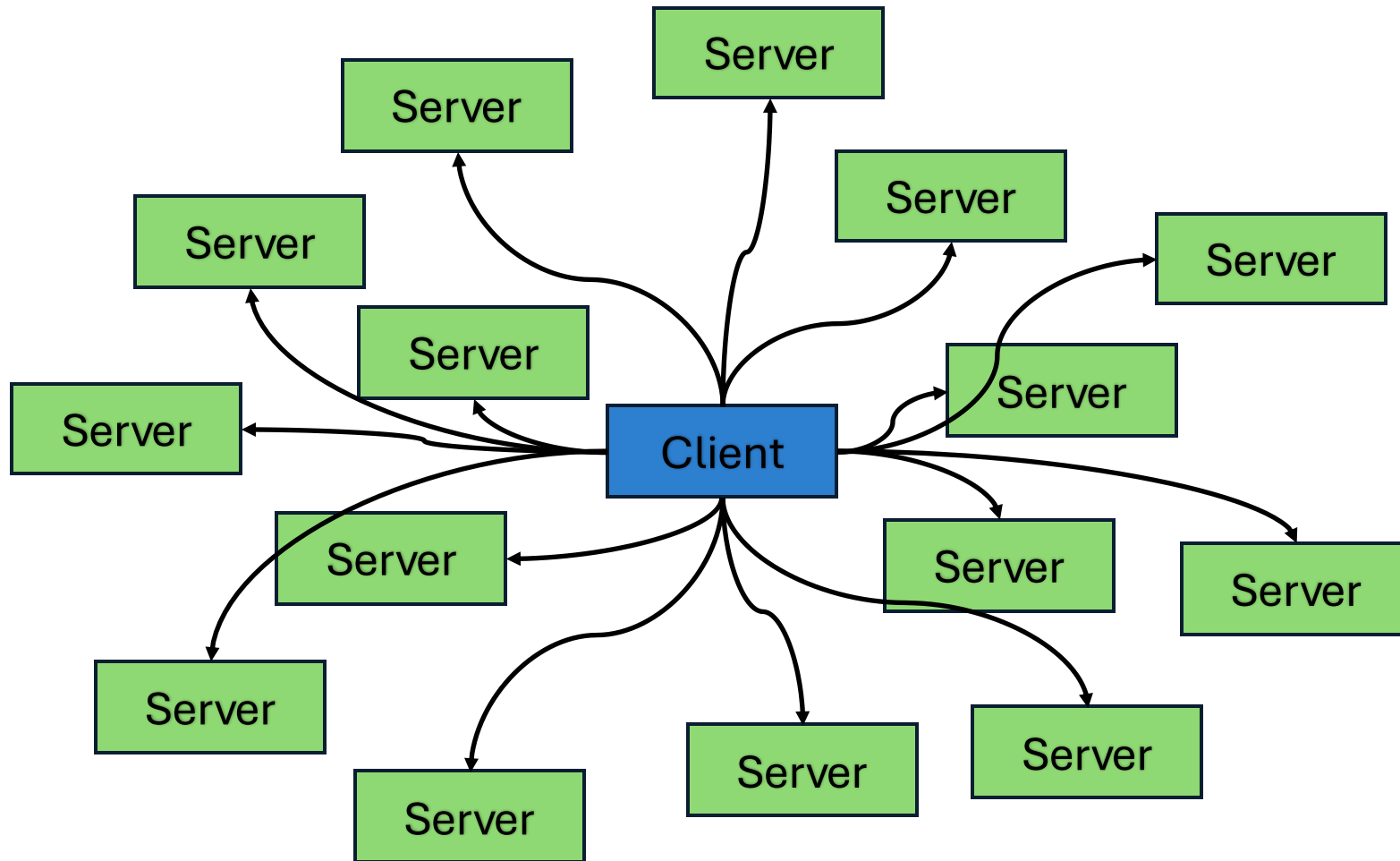


Previous Work – Remote OpenMP Offloading



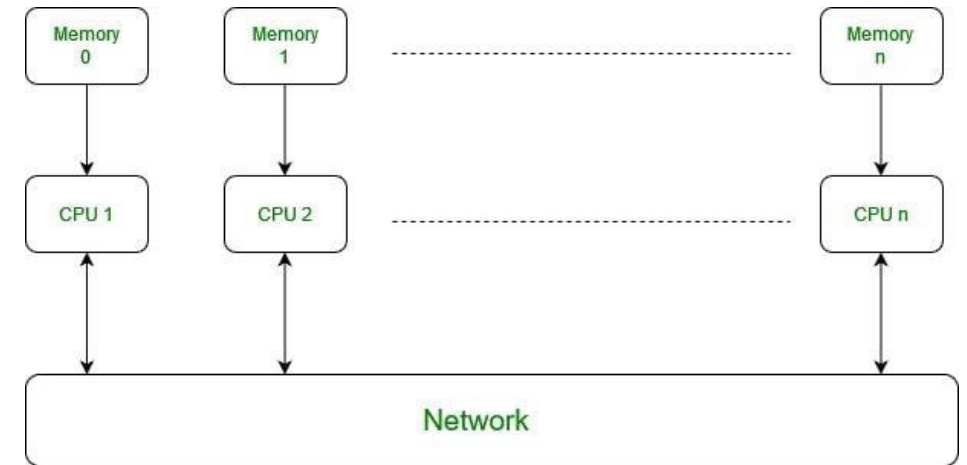
LLVM/OpenMP remote offloading workflow for CUDA devices

The “Command Center”



Let's go back to SPMD

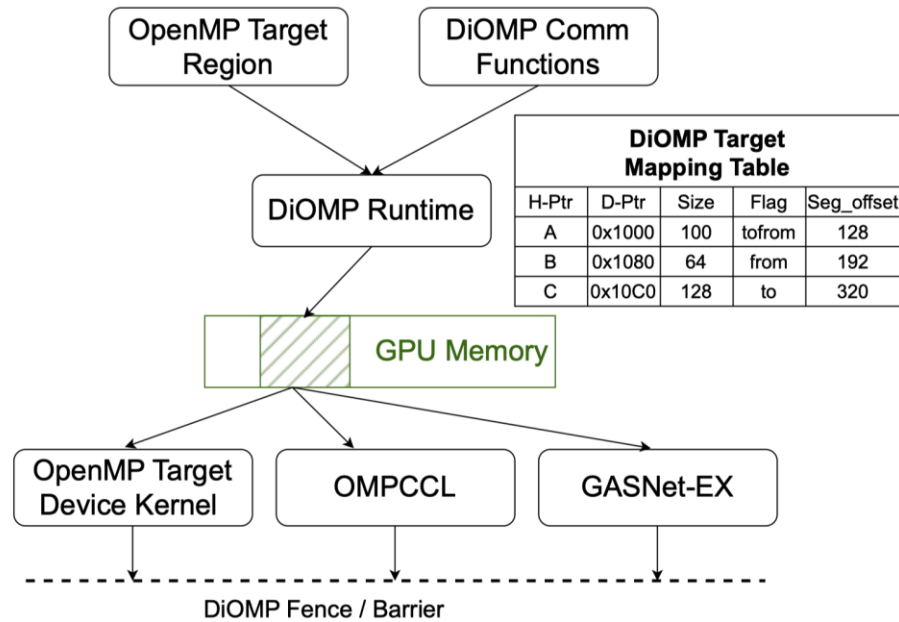
- MPI is a typical SPMD model, as is PGAS.
- In a SPMD model:
 - Data parallelism
 - Inter-node communication required
 - Decentralization



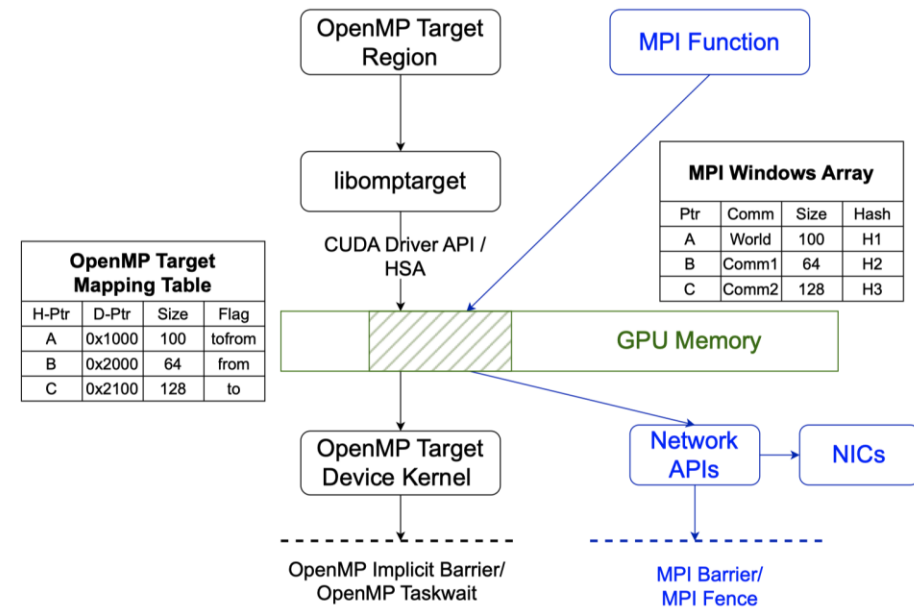
Solution – DiOMP-Offloading

- **Unified Runtime:** A single DiOMP Runtime handles both OpenMP target regions and DiOMP communication functions.
- **Centralized Map Table:** The runtime maintains a single DiOMP target map table, eliminating redundancy.
- **Coordinated Components:** Unifies the coordination of OpenMP device kernels, IPC(intra-node P2P), GASNet-EX (inter-node P2P), OMPCCL (collectives).
- **Unified Synchronization:** Uses a single DiOMP fence or barrier to ensure data consistency.

DiOMP-Offloading VS MPI + OpenMP Target



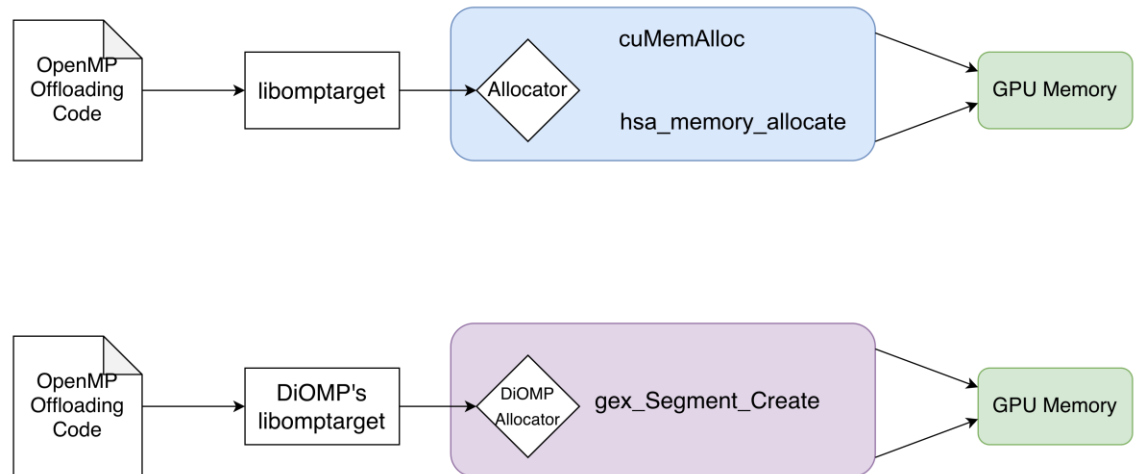
DiOMP-Offloading



MPI RMA + OpenMP Target Offloading

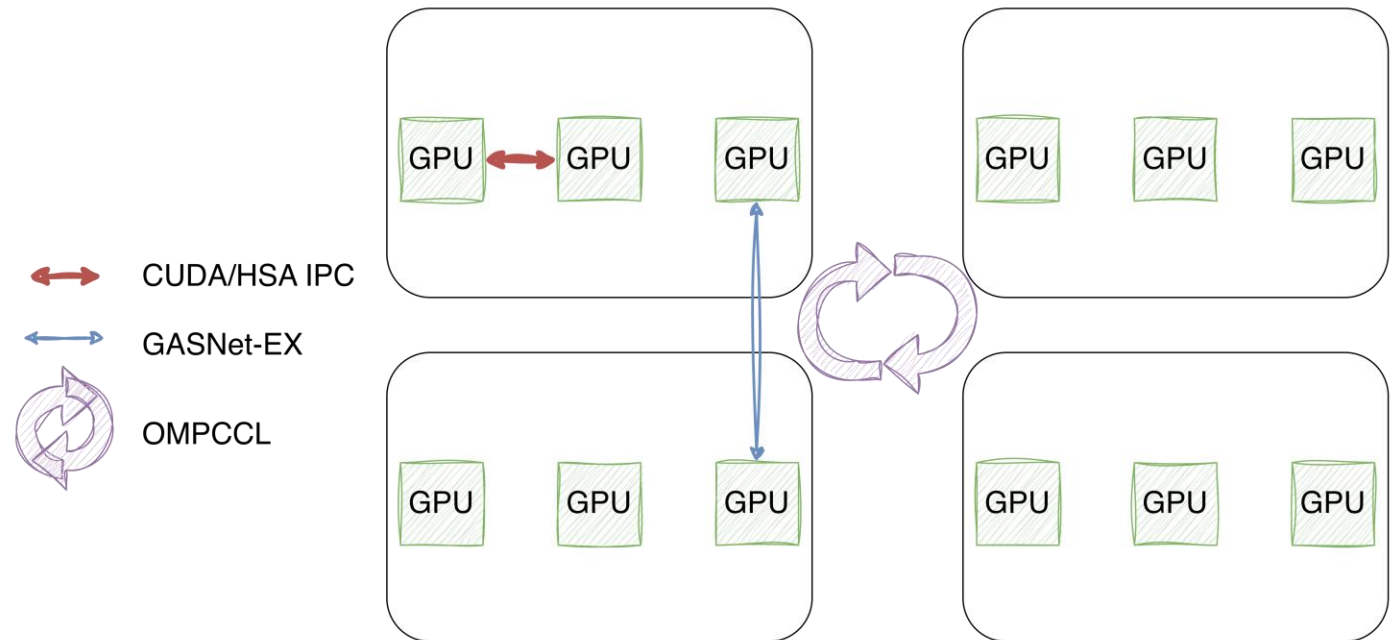
Implementation – Allocator

- Redefine the device memory allocator inside libomptarget.
- Allocated (implicitly or explicitly) memory will be loaded on global space by default.
- RMA function could detect Host or Device pointer and transform automatically.



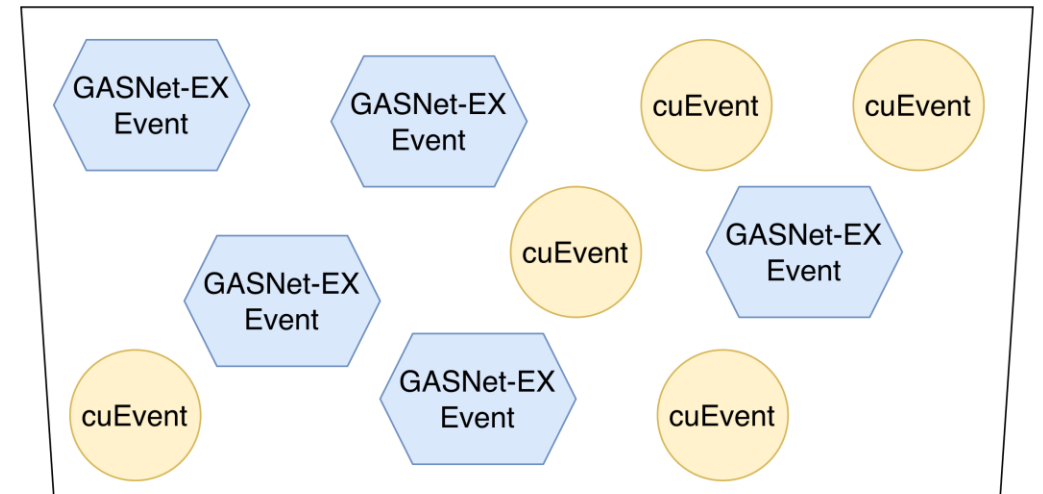
Implementation – Locality-aware Communication

- IPC for intra-node P2P
- GASNet-EX for inter-node P2P
- OMPCCL for collective



Implementation – Mixed Event Polling

- IPC for intra-node P2P – cuEvent/HSA_event
- GASNet-EX for inter-node P2P – gex_event

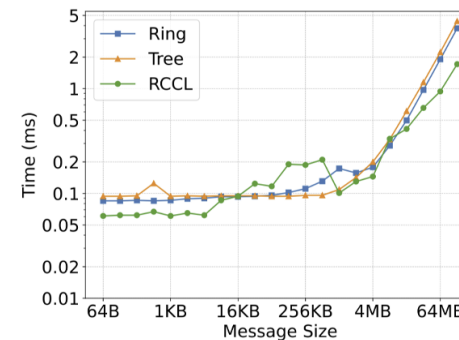
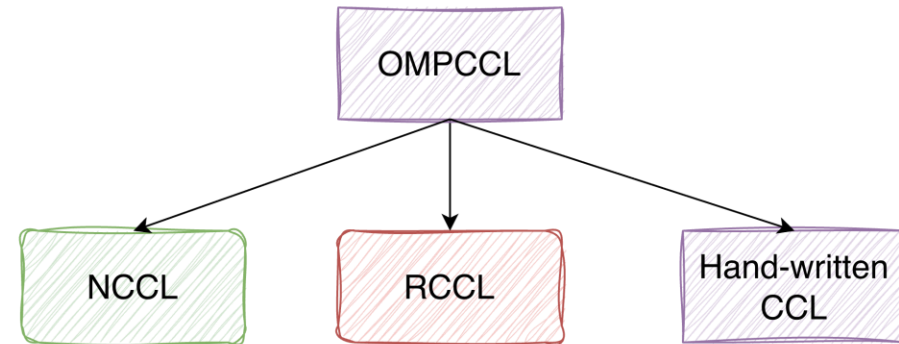


Implementation – OMPCCL

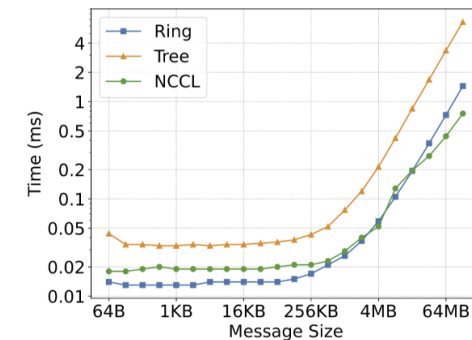
OMPCCL:

Wrapper of NCCL and RCCL

Hand-written Collective Algorithm* is for intra-node collective.



(a) 8 × AMD MI250X



(b) 4 × NVIDIA A100

Ring & Tree are “Hand-written”

*Shan, B., Araya-Polo, M., Doerfert, J., Chapman, B. (2026). Discussion of Device-Device Collective Communication in OpenMP Target Offloading. In: Yan, Y., Klemm, M., de Supinski, B.R., Saule, E., Klíkenberg, J., Pophale, S. (eds) OpenMP: Balancing Productivity and Performance Portability. IWOMP 2025. Lecture Notes in Computer Science, vol 16123. Springer, Cham.

DiOMP vs Others

DiOMP Offloading: Get or Memcpy + Fence

```
// 1. Directly issue a non-blocking get operation
ompx_get( ... );
// or memcpy
ompx_target_memcpy_async( ... );

// 2. Synchronize to ensure the operation completes
ompx_fence();
```

MPI Two-Sided: Request + Pragma + Isend/Recv + Wait

```
if (my_rank == sender_rank) {
    // Asynchronous send returns a request handle
    MPI_Request req;

    #pragma omp target use_device_ptr(dst_ptr)
    MPI_Isend( ... , &req);

    MPI_Wait(&req, MPI_STATUS_IGNORE);
} else if (my_rank == receiver_rank) {
    MPI_Request req;

    #pragma omp target use_device_ptr(dst_ptr)
    MPI_Irecv( ... , &req);

    MPI_Wait(&req, MPI_STATUS_IGNORE);
}
```

DiOMP vs Others

DiOMP Offloading: Get or Memcpy + Fence

```
// 1. Directly issue a non-blocking get operation
ompx_get( ... );
// or memcpy
omp_target_memcpy_async( ... );

// 2. Synchronize to ensure the operation completes
ompx_fence();
```

MPI One-Sided: Win Create + Fence + Pragma + Get + Fence

```
// 1. Setup Window: A window must be created.
MPI_Win_create( ... , &win);

// 2. Communication & Synchronization: Requires Fence/Lock.
MPI_Win_fence(0, win);

// 3. Ensure MPI can access the device pointer.
#pragma omp target use_device_ptr(dst_ptr)
MPI_Get( ... , win);

MPI_Win_fence(0, win); // End epoch

// 4. Cleanup
MPI_Win_free(&win);
```

DiOMP vs Others

DiOMP Offloading: Get or Malloc + Fence

```
// 1. Directly issue a non-blocking get operation
ompx_get( ... );
// or memcpy
omp_target_memcpy_async( ... );

// 2. Synchronize to ensure the operation completes
ompx_fence();
```

OpenSHMEM: Malloc + Update(From) + Get + Sync + Update(To)

```
// 1. Allocate symmetric memory using a special function
src_ptr = shmem_malloc(size);

// 2. Copy data from device to host
#pragma omp target data update(from: ... )

// 3. Communication: Performed between host memories
shmem_get( ... );

// 4. Synchronize SHMEM operation
shmem_quiet();

// 5. Copy data from host back to device
#pragma omp target data update(to: ... )
```

Experiment Setup

NERSC Perlmutter:

NVIDIA A100 * 4, HPE Slingshot 11, Cray MPICH

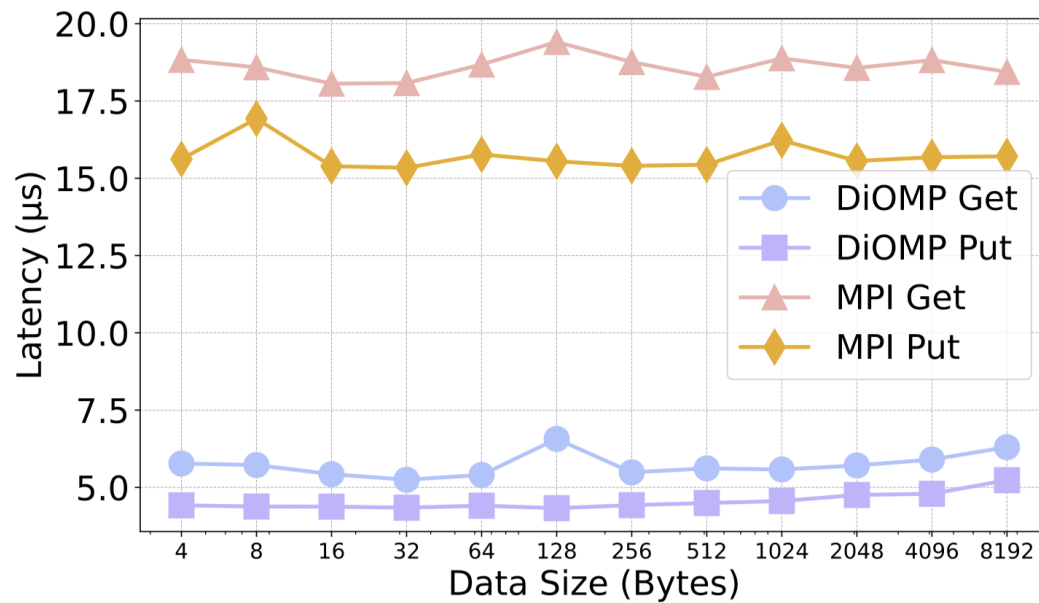
LC(Livermore Computing) Tioga:

AMD MI250X * 4, HPE Slingshot 11, Cray MPICH

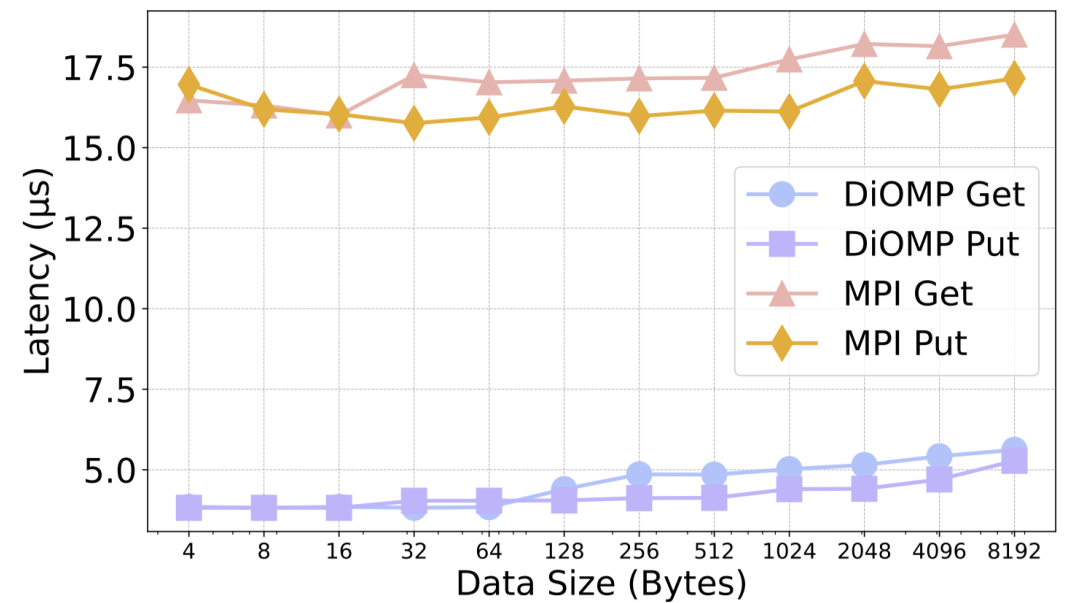
TotalEnergies Maple:

NVIDIA Grace Hopper(GH200), InfiniteBand NDR, OpenMPI

Point-to-Point Latency

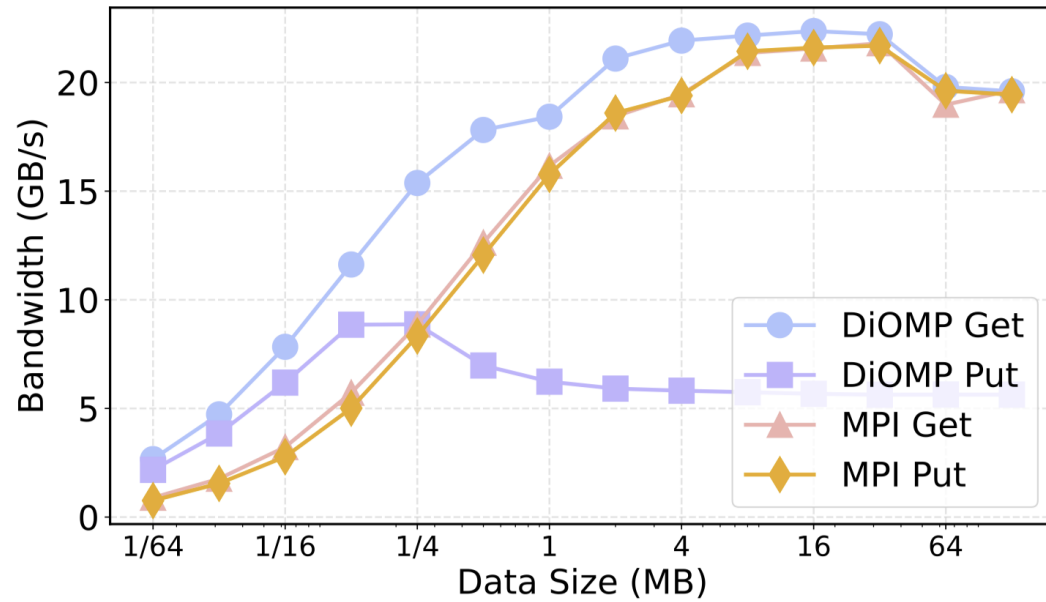


(a) Slingshot 11 + A100

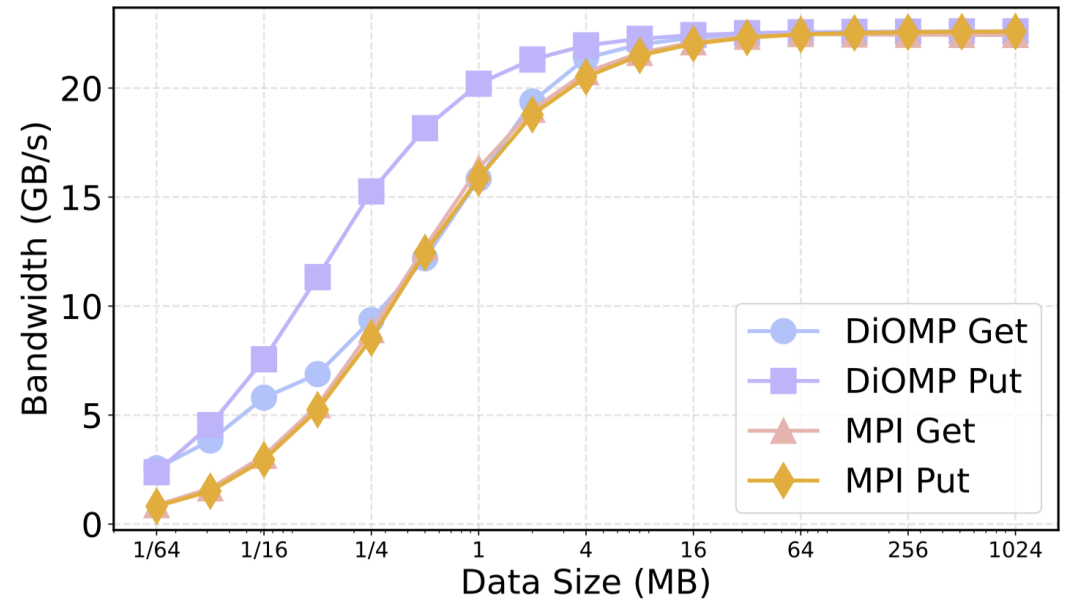


(b) Slingshot 11 + MI250X

Point-to-Point Bandwidth

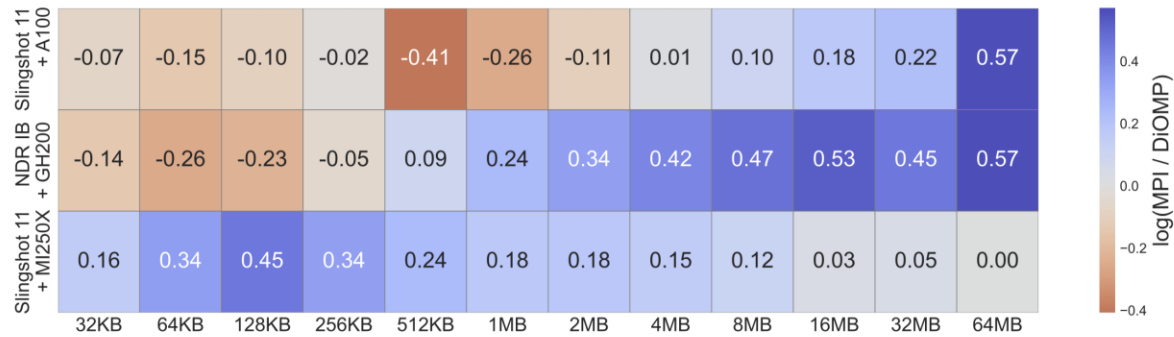


(a) Slingshot 11 + A100*

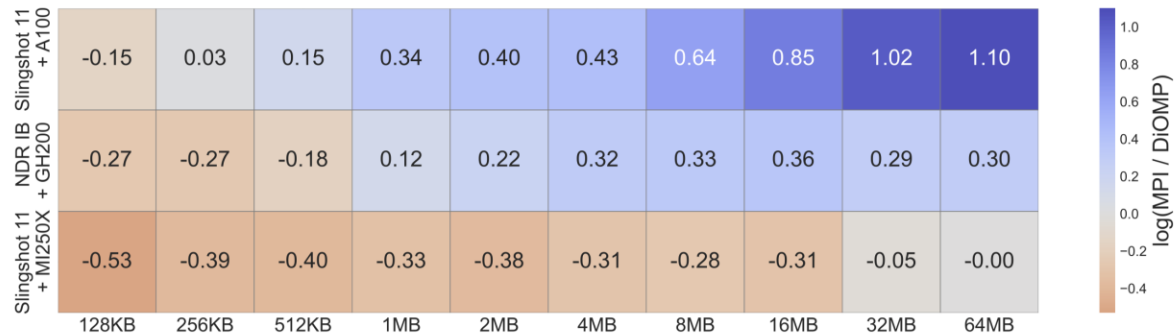


(b) Slingshot 11 + MI250X

Collective



(a) Broadcast

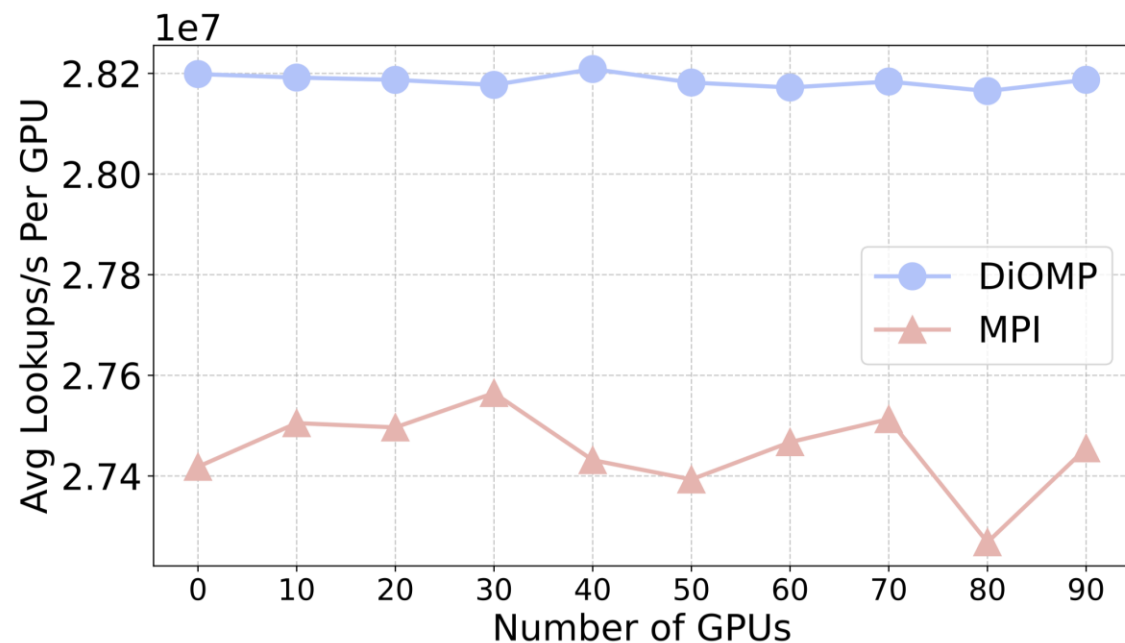


(b) ReduceAll(Sum)

Logarithmic performance ratio ($\log_{10}$) of MPI vs. DiOMP collective latency.

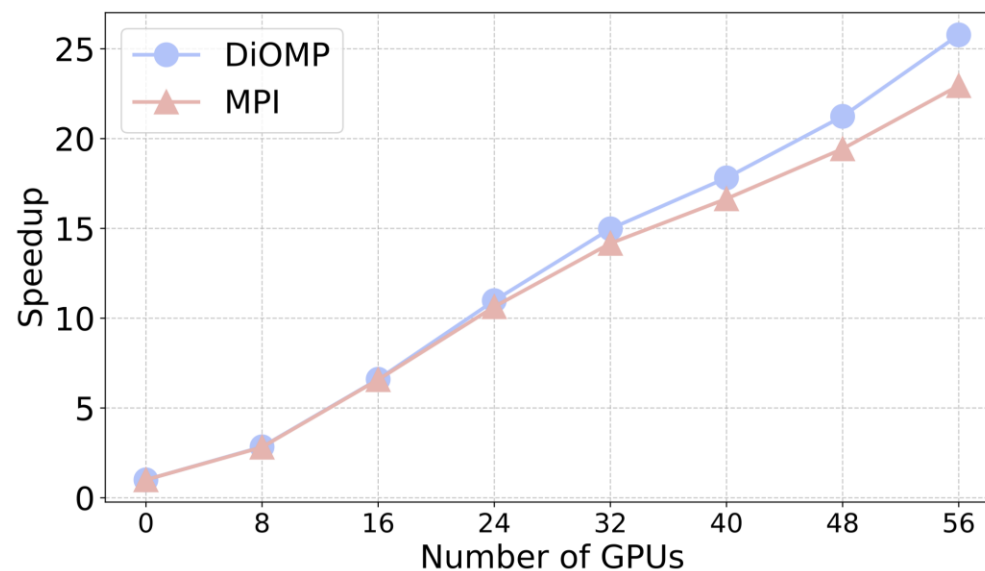
Positive values (cool colors) indicate DiOMP is faster; negative values (warm colors) indicate MPI is faster.

Benchmark - XSbench and Matrix Multiplication



(a) Slingshot 11 + A100

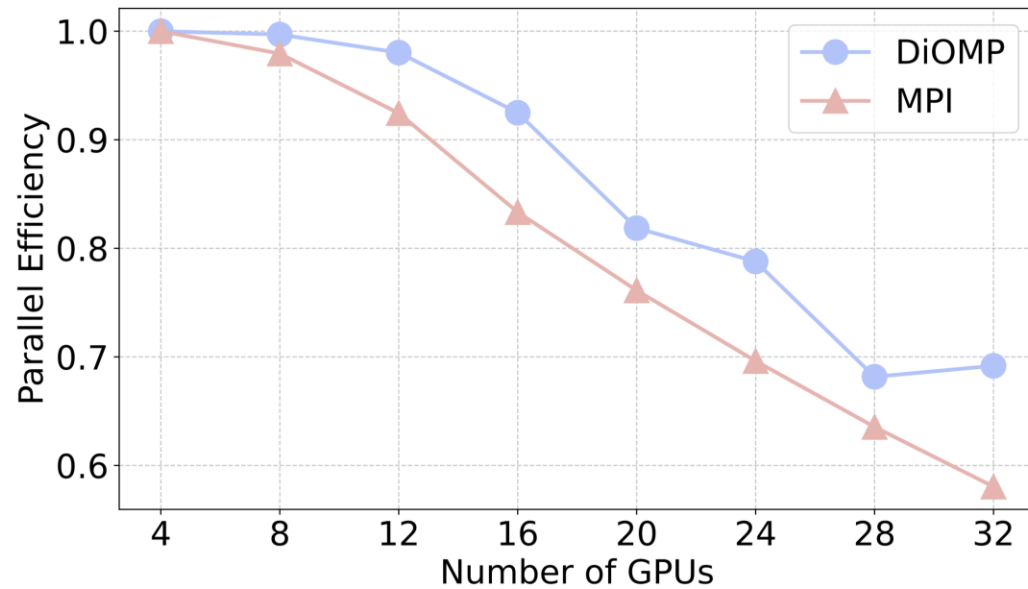
XSbench on Perlmutter.
Higher is better.



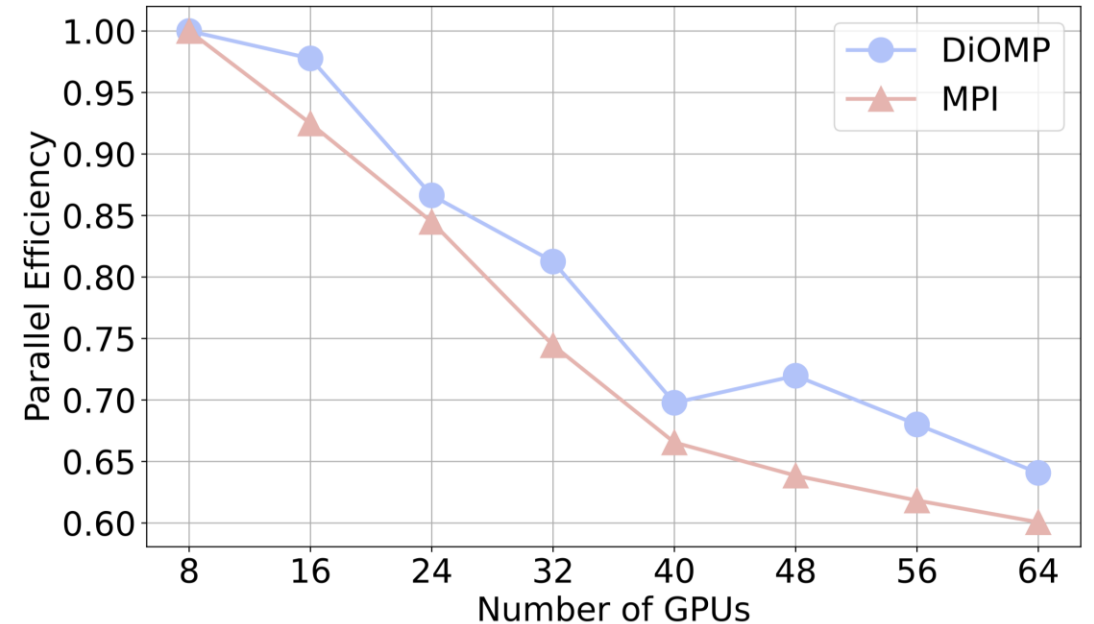
(b) Slingshot 11 + MI250X

Matrix Multiplication
Higher is better.

Minimod – Stencil Computation Proxy App



(a) Slingshot 11 + A100



(b) Slingshot 11 + MI250X

Higher is better.

Future Work

- GPU-Initiated Communication (GDA)
- Task-based OpenMP

Conclusion

- DiOMP-Offloading provides a **unified, portable, and programmable** framework for heterogeneous distributed computing. It achieves superior performance and higher productivity over MPI + OpenMP in most scenarios.

Acknowledgement



Stony Brook
University



Q&A

Thanks!

Contact: *baodi.shan@stonybrook.edu*